

Previous question solve

20-21:

1(a) Define Computer Organization and Computer Architecture. What is the difference between them? (2 marks)

Computer Organization:

👉 এটা বলে কীভাবে *system* কাজ করে — অর্থাৎ, control signal, data flow, memory access ইত্যাদি নিয়ে কাজ করে।

(“How it operates internally.”)

Computer Architecture:

👉 এটা বলে *system* কী করে — অর্থাৎ, design, instruction set, structure, programmer যা দেখতে পায়।

(“What the system is designed to do.”)

Aspect	Computer Architecture	Computer Organization
Focus	Logical design, instruction set	Hardware implementation, circuits
Example	Addressing modes, ISA	Control signals, buses

1(b) What are the key concepts of Von Neumann architecture? Define hardware and software programming. (5 marks)

Von Neumann Architecture Concepts:

1. **Single memory** for both **data and instructions**.
2. **Sequential execution** — one instruction at a time.
3. **Stored program concept** — program stored in memory.
4. **ALU, Control Unit, Memory, I/O units** are main parts.
5. **Buses (data, address, control)** used to transfer data.

🧠 This is the basic design used in most modern computers.

Hardware Programming:

👉 Writing instructions in **machine code** or **assembly** that directly control hardware components.

♦ Example: Writing MOV A, B in Assembly.

Software Programming:

👉 Writing programs using **high-level languages** (C, Python, Java) that are later translated into

machine code.

- ♦ Example: $A = B + C$; in C language.
-

1(c) Define Interrupts. What are the approaches for handling multiple interrupts? Illustrate and explain. (5 marks)

Interrupt:

👉 An **interrupt** is a signal sent to the CPU that **temporarily stops** the current execution and jumps to an **Interrupt Service Routine (ISR)** to handle an event.

Example: Keyboard input, Timer, or I/O request.

Approaches for Handling Multiple Interrupts:

1. Disabling Interrupts:

- CPU disables all other interrupts while one is being serviced.
- Simple but may miss urgent interrupts.

2. Fixed Priority:

- Each interrupt has a fixed priority level.
- High-priority interrupt can interrupt low-priority ISR.

3. Daisy Chaining (Hardware Priority):

- Devices connected in a chain; the first device has the highest priority.
- If first device doesn't request, control passes to next.

4. Polling Method:

- CPU checks each device in sequence to find which requested interrupt.
 - Slower, but simple to implement.
-

Illustration (Example):

Keyboard Interrupt → Priority 1

Printer Interrupt → Priority 2

Disk Interrupt → Priority 3

If both Keyboard and Disk send interrupt signals,

👉 CPU will handle **Keyboard** first (highest priority).

1(d) What are the steps of the instruction cycle for the instruction "SUB B, A," which stores the difference of memory locations B and A into location A? (2 marks)

❖ 1(d) Instruction Cycle of “SUB B, A” (2 marks)

Instruction: SUB B, A $\rightarrow$ A = A - B

Steps:

1. **Fetch** – Memory থেকে instruction নেওয়া।
 2. **Decode** – Subtraction identify করা ও operand A, B নির্ধারণ।
 3. **Fetch Operands** – A, B এর মান register এ load করা।
 4. **Execute** – ALU করে A = A - B।
 5. **Store Result** – ফলাফল memory location A তে store করা।
-

2(a) Define computer instructions and describe their types. (3 marks)

Definition:

👉 A **computer instruction** is a command given to the CPU to perform a specific task such as data transfer, arithmetic, logic, or control operations.
It is written in **machine code** or **assembly language**.

Types of Instructions:

1. **Data Transfer Instructions:** Move data between registers, memory, or I/O.
➤ Example: MOV A, B
 2. **Arithmetic Instructions:** Perform mathematical operations.
➤ Example: ADD A, B
 3. **Logical Instructions:** Perform logic operations like AND, OR, NOT.
➤ Example: AND A, B
 4. **Control Instructions:** Change the sequence of execution (like jump, call).
➤ Example: JMP LABEL
 5. **Input/Output Instructions:** Handle data transfer between CPU and I/O devices.
➤ Example: IN, OUT
-

2(b) Where Zero-address instructions are applicable and how? Explain. (2 marks)

Answer:

👉 Stack-based machine এ use হয়। Operands stack থেকে নেয়।
No explicit operand লাগে না।

Example:

PUSH A

PUSH B

ADD $\leftarrow$ top দুই value add করে

Used in stack computers like JVM, HP calculators.

2(c) What are logical, arithmetic, and cyclic (rotate) shifts? Apply a 4-bit logical right shift, logical left shift, arithmetic right shift, arithmetic left shift, right rotate, left rotate to the bits 10100110. (5 marks)

Definition:

1 Logical Shift

👉 Logical shift মানে হলো — বিটগুলোকে ডান বা বামে সরানো, আর ফাঁকা জায়গাগুলোতে 0 বসানো।

♦ Sign bit (MSB)-এর কোনো মানে এখানে থাকে না, কারণ এটা শুধু bit shifting-এর জন্য ব্যবহৃত হয়।

2 Arithmetic Shift

👉 Arithmetic shift ব্যবহৃত হয় signed numbers (two's complement) এর ক্ষেত্রে। এখানে sign bit (MSB) ধরে রাখা হয় যেন সংখ্যা positive/negative ঠিক থাকে।

3 Cyclic (Rotate) Shift

👉 Rotate মানে হলো শেষ দিকের bit আবার শুরুতে ফিরে আসে (বা উল্টোভাবে)। কোনো bit হারায় না, শুধু ঘুরে যায়।

আমাদের মূল 8-bit সংখ্যা:

10100110

1 4-bit Logical Right Shift ($\gg 4$)

👉 সব বিট ৪ ঘর ডানে সরবে, বাম পাশে ০ বসবে।

Original: 10100110

Shifted : 00001010

✅ Result: 00001010

2 4-bit Logical Left Shift ($\ll 4$)

👉 সব বিট ৪ ঘর বামে সরবে, ডান পাশে ০ বসবে।

Original: 10100110

Shifted : 01100000

✓ **Result: 01100000**

✖ 3 4-bit Arithmetic Right Shift

👉 **Sign bit (MSB)** অপরিবর্তিত থাকবে, বাকি বিট ৪ ঘর ডানে সরবে।
(বাম পাশে sign bit পূরণ হবে)

Original: 10100110

Sign bit: 1

Shifted : 11111010

✓ **Result: 11111010**

✖ 4 4-bit Arithmetic Left Shift

👉 প্রায় Logical Left Shift এর মতোই — শুধু overflow হলে sign বদলাতে পারে।
এখানে বামে ৪ ঘর সরিয়ে ডান পাশে ০ বসাই।

Original: 10100110

Shifted : 01100000

✓ **Result: 01100000**

🌀 5 4-bit Right Rotate

👉 শেষের ৪ বিট প্রথমে চলে আসবে, প্রথমের ৪ বিট শেষে যাবে।

Original: 1010 | 0110

Rotate Right 4 → 0110 | 1010

✓ **Result: 01101010**

🌀 6 4-bit Left Rotate

👉 প্রথমের ৪ বিট শেষে চলে যাবে, শেষের ৪ বিট প্রথমে আসবে।

Original: 1010 | 0110

Rotate Left 4 → 0110 | 1010

✓ **Result: 01101010**

(দেখো, 8-bit সংখ্যায় 4-bit rotate left = rotate right একই ফল দেয়)

✓ **Final Summary Table**

Operation	Result (8-bit)
Logical Right Shift (4-bit)	00001010
Logical Left Shift (4-bit)	01100000
Arithmetic Right Shift (4-bit)	11111010
Arithmetic Left Shift (4-bit)	01100000
Right Rotate (4-bit)	01101010
Left Rotate (4-bit)	01101010

2(d) What is a procedure, and why is it necessary? Illustrate and explain the procedure mechanism. (4 marks)

Definition:

👉 A **procedure** (or subroutine/function) is a block of code that performs a specific task and can be called from anywhere in the program.

Necessity / Advantages:

1. **Code Reusability:** One procedure can be used many times.
2. **Modularity:** Breaks a big program into smaller parts.
3. **Easy Debugging and Maintenance.**

Procedure Mechanism:

1. **Call:** CPU saves the **return address** (next instruction) in the stack and jumps to the procedure.
2. **Execute:** Procedure instructions run.
3. **Return:** Control comes back to the saved return address.

Example:

CALL ADDITION ; call procedure

...

ADDITION: ; procedure starts

ADD A, B

RET ; return to main program

3(a) What is an I/O module, and why is it necessary? (3 marks)

Definition:

I/O Module কী?

👉 এটি একটা hardware অংশ যা CPU এবং বাইরে থাকা devices (যেমন keyboard, mouse, printer) এর মধ্যে যোগাযোগ করায়।

কারণে লাগে / কাজগুলো:

1. Device Communication:

CPU এবং external devices-এর মধ্যে data path তৈরি করে।

যেমন: Keyboard থেকে typed key CPU পর্যন্ত পৌঁছানো।

2. Data Conversion:

Device-এ signal হতে পারে analog বা serial, CPU binary বোঝে।

I/O module এটাকে পরিবর্তন করে CPU-compatible করে।

3. Control / Management:

Device ready কি না, error আছে কি না, timing ঠিক আছে কি না—সব দেখাশোনা করে।

4. CPU Load কমায়:

Slow device direct handle করলে CPU wait করতে হবে। I/O module handle করলে CPU অন্য কাজ করতে পারে।

Example:

- তুমি keyboard-এ key চাপছো।
 - I/O module সেটা input নেয় → binary তে convert করে → CPU-কে দেয়।
-

3(b) What are the differences among Programmed I/O, Interrupt-driven I/O, and Direct Memory Access (DMA)? Describe with flowcharts. (5 marks)

🧠 1 Programmed I/O (Polling method)

👉 এখানে CPU নিজেই বারবার দেখে ডিভাইস (যেমন কীবোর্ড, প্রিন্টার) রেডি কিনা। ডিভাইস রেডি না থাকলে CPU অপেক্ষা করে — কিছুই করতে পারে না।

📖 Example:

তুমি কারো ফোন আসার অপেক্ষায় ফোনটা তাকিয়ে তাকিয়ে দেখছ — অন্য কিছু করছ না।

♦ Process:

1. CPU → I/O device-কে বলে: "তুমি data পাঠাও।"
2. CPU বারবার চেক করে: "রেডি হয়েছে?"

3. রেডি হলে CPU ডেটা নেয়।

4. তারপর পরের কাজ করে।

🕒 **Problem:** CPU সময় নষ্ট করে (busy waiting)।

⚡ 2 Interrupt-driven I/O

👉 এখানে ডিভাইস রেডি হলে নিজেই CPU-কে interrupt পাঠায়।
ততক্ষণ CPU অন্য কাজ করতে পারে।

📖 **Example:**

তুমি ফোনটা রেখে নিজের কাজ করছ। ফোন বাজলেই (interrupt) তুমি ধরো।

♦ **Process:**

1. CPU I/O-কে বলে: “তুমি রেডি হলে ডাক দিও।”
2. CPU নিজের অন্য কাজ করে।
3. ডিভাইস রেডি হলে interrupt পাঠায়।
4. CPU কাজ থামিয়ে I/O হ্যান্ডেল করে, তারপর আগের কাজ চালায়।

✅ **Benefit:** CPU সময় বাঁচায়, parallel কাজ হয়।

🚀 3 Direct Memory Access (DMA)

👉 এখানে ডেটা I/O ডিভাইস ↔ মেমোরির মধ্যে সরাসরি ট্রান্সফার হয়, CPU-কে পাশ কাটিয়ে।

📖 **Example:**

তুমি বন্ধুকে বললে, “এই ফাইলটা কপি করে দে” — তুমি নিজে না করে বন্ধু (DMA controller) করে দিল।

♦ **Process:**

1. CPU শুধু DMA controller-কে বলে: “এই ডেটা ওখানে পাঠাও।”
2. DMA নিজে ডেটা ট্রান্সফার করে।
3. কাজ শেষ হলে CPU-কে interrupt পাঠায়।

✅ **Benefit:** CPU ফ্রি থাকে, দ্রুত ডেটা ট্রান্সফার হয়।

🔧 **Use:** বড় ডেটা ট্রান্সফারে যেমন — হার্ডডিস্ক, মেমরি কপি ইত্যাদি।

📊 Comparison Table

Feature	Programmed I/O	Interrupt-driven I/O	DMA
Who controls transfer?	CPU	CPU (with interrupt)	DMA Controller
CPU waiting?	Always busy	Not busy	Not busy
Speed	Slow	Faster	Fastest
Use case	Small devices	Medium-speed devices	High-speed data transfer
CPU load	High	Medium	Very Low

✳ সহজ মনে রাখার উপায়:

ধরো তুমি ডেলিভারি দিতে বললে

উদাহরণ

তুমি নিজেই হাতে হাতে দিচ্ছে

Programmed I/O

ডেলিভারি বয় রেডি হলে ফোন দিচ্ছে

Interrupt-driven I/O

ডেলিভারি বয় নিজেই দিয়ে শেষে জানাচ্ছে DMA

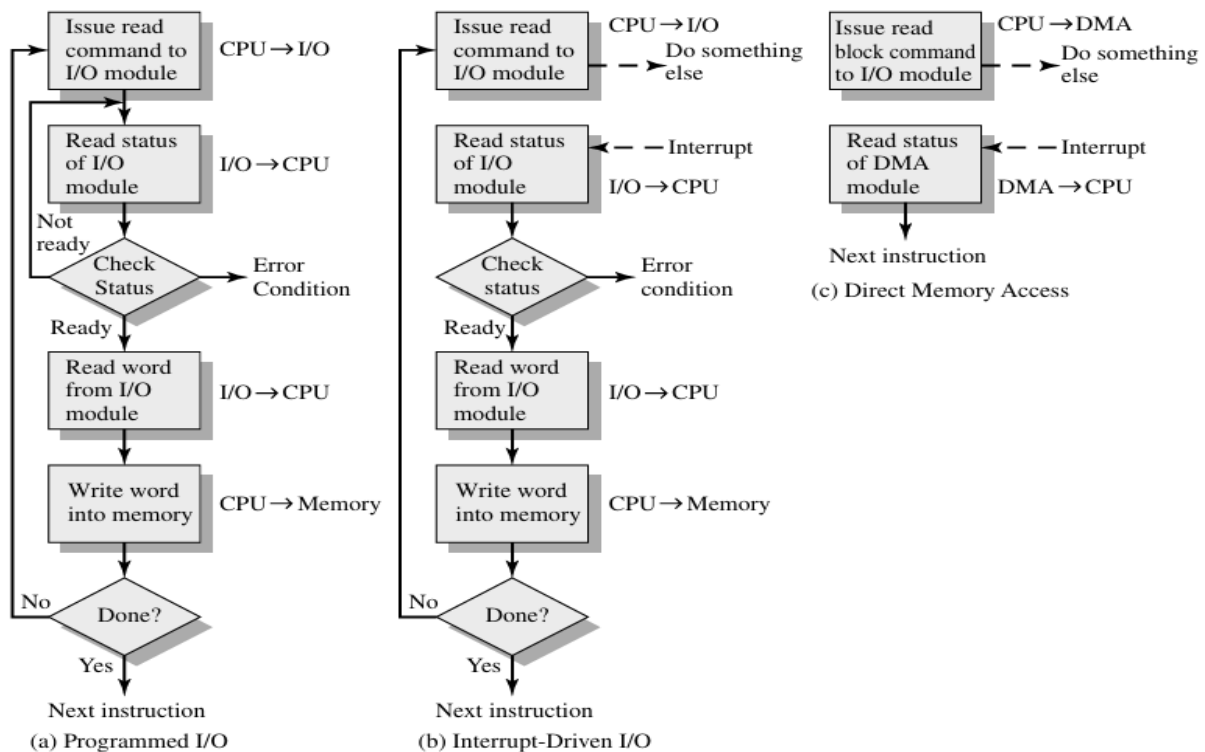


Figure 7.4 Three Techniques for Input of a Block of Data

3(c) How does a DMA module transfer control to the processor after its task? Does it interrupt the processor? Define cycle stealing in DMA. (2 marks)

Answer:

✅ DMA কাজ শেষ হলে CPU কে interrupt পাঠায় জানাতে যে কাজ শেষ।

👉 So yes, DMA interrupts the processor.

Cycle Stealing:

DMA এক memory cycle এর জন্য CPU থেকে bus নিয়ে নেয় data transfer এর জন্য।

CPU সাময়িক pause হয় 1 cycle এর জন্য — পরে আবার কাজ চালায়।

একদম সহজভাবে বুঝি 🙌

💡 **Cycle Stealing (in DMA) মানে কী?**

👉 যখন DMA controller ডেটা ট্রান্সফার করে, তখন তার memory access করতে হয় — কিন্তু memory bus তো CPU-ও ব্যবহার করে!

তাই DMA বলে —

“CPU ভাই, একটু সময়ের জন্য bus টা দাও, আমি শুধু ১টা memory cycle ব্যবহার করবো।” 😊

⚙️ **What happens:**

1. CPU কাজ করছে 🧠
 2. DMA এক মুহূর্তের জন্য CPU থেকে bus “চুরি” নেয় (steal করে)।
 3. ঐ সময় DMA ডেটা ট্রান্সফার করে (I/O ↔ Memory)।
 4. ট্রান্সফার শেষে DMA bus ফিরিয়ে দেয় CPU-কে।
 5. CPU আবার আগের কাজ চালায়।
-

🕒 **Result:**

- CPU এক cycle এর জন্য pause হয়, পুরোপুরি বন্ধ হয় না।
 - তাই এটাকে বলা হয় Cycle Stealing — কারণ DMA “একটা cycle” ধার নেয়।
-

📖 **Example দিয়ে বুঝো:**

ধরো তুমি (CPU) ক্লাসে লিখছ, হঠাৎ বন্ধু (DMA) বলে:

“এক সেকেন্ড কলমটা দে, একটা শব্দ লিখে দিই!”

তুমি ১ সেকেন্ড থামলে, সে লিখে কলম ফেরত দিল।

তুমি আবার লিখা চালিয়ে গেলে।

👉 এটাই হলো Cycle Stealing।

3(d) Give the features of CISC and Core i7 in the perspective of computer architecture and organization. (4 marks)

CISC (Complex Instruction Set Computer):

1. অনেক complex instruction
2. এক instruction এ multiple task
3. Assembly-friendly
4. Slow execution but short code
Example: Intel 8086, Pentium

Core i7 (Modern Intel Processor):

1. CISC ভিত্তিক কিন্তু ভিতরে RISC micro-ops
2. Multi-core + Hyper-threading
3. L1/L2/L3 Cache
4. Branch prediction, pipelining, GPU integrated
💡 Result → High performance modern processor.

অবশ্যই! 😊 চলো একদম সহজভাবে CISC vs Core i7 তুলনা করি computer architecture & organization এর দিক থেকে 👉

1 CISC (Complex Instruction Set Computer)

- **Instruction set:** অনেকগুলো জটিল এবং বড় কমান্ড থাকে।
→ একেকটা instruction অনেক কাজ করতে পারে।
- **Programming:** assembly প্রোগ্রাম করা সহজ।
- **Execution:** এক instruction-এ অনেক operation → কিন্তু একেকটা instruction ধীর।
- **Code size:** ছোট কোড, কারণ এক instruction অনেক কাজ করে।
- **Example:** Intel 8086, Pentium (পুরনো processors)

সারসংক্ষেপ:

CISC = “Instruction-heavy, slower execution, compact code”

2 Core i7 (Modern Intel processor)

- **Based on CISC internally:** কিন্তু ভিতরে instruction কে RISC-like micro-operations এ ভাগ করে execute করে।
- **Performance:**
 - Multi-core → একসাথে অনেক কাজ করতে পারে
 - Hyper-threading → একটি core multiple threads handle করে
 - Pipelining → এক instruction চালাতে গেলে next instruction শুরু হয়ে যায়
- **Cache:** L1, L2, L3 cache → দ্রুত ডেটা access
- **Advanced features:** branch prediction, out-of-order execution, integrated GPU

সারসংক্ষেপ:

Core i7 = “Modern CISC with RISC-like execution, high speed, multi-core, advanced optimizations”

✅ Comparison Table (Simple Version)

Feature	CISC	Core i7
Instruction Set	Complex, many operations	CISC based, internally uses RISC ops
Execution	Slower per instruction	Very fast (pipelining, out-of-order)
Code size	Compact	Medium (depends on micro-ops)
CPU Features	Basic	Multi-core, Hyper-threading, Cache, Branch prediction
Example	Intel 8086, Pentium	Intel Core i7

💡 Remember:

CISC = পুরনো traditional design

Core i7 = আধুনিক processor, CISC এর evolution + speed optimization

4(a)

Step 0: Given Opcodes

Operation	Opcode
Load from Memory	1110
Store to Memory	0001
Add	0101
Subtract	1011
Store to I/O	0110
Load from I/O	1101

Program:

- 1 Load AC from device 99
 - 2 Add contents of memory 555
 - 3 Store AC to memory 666
-

Step 1: Convert addresses to 12-bit binary & hex

Address 12-bit binary Hex

99 000001100011 063
555 001000101011 22B
666 001010011010 29A

Step 2: Combine Opcode + Address → 16-bit instruction

Inst Instruction (Binary) Hex Meaning

1 1101 000001100011 D063 Load AC from I/O 99
2 0101 001000101011 522B Add memory[555] to AC
3 0001 001010011010 129A Store AC to memory 666

Step 3: Execution (Fetch → Decode → Execute)

1 Load AC from I/O 99 (D063)

- **Fetch:** CPU reads instruction D063 from memory.
- **Decode:** Opcode 1101 → Load from I/O; Address = 99.
- **Execute:** Read data from device 99 → AC = device[99]

2 Add memory[555] (522B)

- **Fetch:** CPU reads 522B.
- **Decode:** Opcode 0101 → ADD; Address = 555
- **Execute:** Read memory[555] → add to AC → $AC = AC + \text{memory}[555]$

3 Store AC to memory[666] (129A)

- **Fetch:** CPU reads 129A.
- **Decode:** Opcode 0001 → STORE; Address = 666
- **Execute:** $AC \rightarrow \text{memory}[666] \rightarrow \text{memory}[666]$ now stores AC value

Easy way to remember:

- **Step 1:** Load → AC gets value from device
- **Step 2:** Add → $AC + \text{memory}$
- **Step 3:** Store → $AC \rightarrow \text{memory}$

✓ AC always holds the “current result” during execution.

4(b) (2 + 2 marks)

(i) Explain RAID-2 (সংক্ষিপ্ত):

- RAID-2 হল ঐতিহাসিক/শিক্ষণধর্মী RAID level যেখানে **bit-level striping** ব্যবহার করা হয়।
- প্রতিটি ডাটা-বাইট (বা শব্দ) প্রতিটি ডিস্কে bit-wise ছড়িয়ে দেওয়া হয় এবং **dedicated Hamming-code parity/disks** ব্যবহার করে ভুল(s) সনাক্ত ও ঠিক করা হয় (error-correcting code)।
- প্রাকৃতিকভাবে, RAID-2-এ দরকার হয়ে পড়ে অনেক ডিস্ক (একাধিক parity/disks for ECC)।
- বাস্তবে RAID-2 খুবই কম ব্যবহৃত কারণ Hamming-ECC টেকনিকগুলো পরে ডিস্ক-কন্ট্রোলার স্তরে করা সহজে এবং আরেকটু দক্ষ RAID schemes (RAID-5 ইত্যাদি) জনপ্রিয় হয়েছে।

(ii) Semiconductor memory-এর ৪টি প্রধান ধরন ও তাদের বৈশিষ্ট্য (সংক্ষিপ্ত):

1. SRAM (Static RAM)

- গঠন: flip-flop-based (6-transistor cell)।

- বৈশিষ্ট্য: দ্রুত (high speed), volatile (power off → ডেটা যায়), দাম বেশি, ব্যবহার: CPU caches (L1, L2)।
2. **DRAM (Dynamic RAM)**
- গঠন: ক্যাপাসিটর + transistor (1T-1C)।
 - বৈশিষ্ট্য: সস্তা, বেশি ঘনত্ব (high density), ধীরতর তুলনায় SRAM, volatile, periodical refresh প্রয়োজন। ব্যবহার: main memory (RAM)।
3. **ROM (Read Only Memory) / Mask ROM**
- গঠন: প্রি-programmed memory; power off-এ ডেটা থাকে (non-volatile)।
 - বৈশিষ্ট্য: read-only বা limited programmability; ব্যবহার: firmware, bootstrap code।
4. **EPROM / EEPROM / Flash (Electrically Programmable ROM)**
- গঠন: floating-gate transistor cell।
 - বৈশিষ্ট্য: non-volatile, electrically erasable/reprogrammable (EEPROM/Flash), slower write/erase than SRAM/DRAM, ব্যবহার: firmware storage, SSD chips (NAND flash) ইত্যাদি।
-

4(c)

Important Addressing Modes (Easy Version)

1. Immediate Addressing

- Operand instruction-এর ভিতরই থাকে (constant value)
- Example: MOV A, #5 → A তে 5 বসানো

2. Direct Addressing

- Instruction-এ ঠিক memory address দেওয়া থাকে
- Example: MOV A, 1000 → memory[1000] থেকে value → A

3. Indirect Addressing

- Instruction-এ এমন address থাকে যেটা বলে memory-তে কোথায় operand আছে
- Example: address 2000 → memory[2000] = 1000 → memory[1000] = actual value

4. Register Addressing

- Operand CPU register-এ থাকে
- Example: ADD A, B → A + B (B register থেকে)

5. Indexed Addressing

- Base address + index register $\rightarrow$ effective address
- Example: MOV A, 2000 + X $\rightarrow$ array element access

6. Stack Addressing (Implicit)

- Operand stack top থেকে আসে
 - Example: PUSH বা POP
-

Memory trick to remember:

- **Immediate:** value already in instruction
 - **Direct:** memory address directly
 - **Indirect:** memory of memory
 - **Register:** CPU register
 - **Indexed:** array / offset
 - **Stack:** LIFO
-

✿ 4(c)(ii) Two's Complement Operations

(a) $-A + B$ (Example)

Let's say $A = 5$, $B = 18$
 $A = 000101$, $B = 010010$
 $-A = \text{invert} + 1 \rightarrow 111011$
Now:

111011

+010010

=1001101 $\rightarrow$ discard carry $\rightarrow 001101 = 0x0D$

✓ Result = +13 (Hex 0x0D)

(b) $-33 - 17$

Range of 6-bit two's complement = $-32 \dots +31$
 $\rightarrow -33$ not representable $\Rightarrow$ **Overflow** ✗

So result not possible within 6-bit limit.

5(a) Cache Mapping

Given:

- Byte addressable memory: $2^{16} = 65,536$ bytes
- Block size = 8 bytes
- Direct mapped cache, 32 lines

Step i: Divide 16-bit memory address

Formulas:

- **Byte number (within block):** $\log_2(\text{block size}) = \log_2(8) = 3 \text{ bits}$
- **Line number (cache line):** $\log_2(\text{number of lines}) = \log_2(32) = 5 \text{ bits}$
- **Tag:** Remaining bits = $16 - (5+3) = 8 \text{ bits}$

✅ Summary:

Field	Bits	Meaning
Tag	8	Identify block uniquely
Line Number	5	Which cache line
Byte Number	3	Which byte in block

Step ii: Cache line for given addresses

- **Address:** 0001 0001 0001 1011

Step 1: Divide into tag-line-byte

- Byte number = last 3 bits $\rightarrow 011 = 3$
- Line number = next 5 bits $\rightarrow 00011 = 3$
- Tag = remaining 8 bits $\rightarrow 00010001$

$\rightarrow$ Cache line = 3

- **Address:** 1101 0000 0001 1101
- Byte number = last 3 bits $\rightarrow 101 = 5$
- Line number = next 5 bits $\rightarrow 00011 = 3$
- Tag = remaining 8 bits $\rightarrow 11010000$

$\rightarrow$ Cache line = 3

দুটো address একই cache line-এ map হবে, তবে tag ভিন্ন $\rightarrow$ conflict miss হতে পারে

Step iii: Other bytes in the block

- Address: 0001 1010 0001 1010
- Block size = 8 bytes $\rightarrow$ block-aligned addresses = last 3 bits = 0–7
- **Block start address:** clear last 3 bits $\rightarrow$ 0001 1010 0001 1000
- **Block bytes:**

0001 1010 0001 1000

0001 1010 0001 1001

0001 1010 0001 1010 $\leftarrow$ given

0001 1010 0001 1011

0001 1010 0001 1100

0001 1010 0001 1101

0001 1010 0001 1110

0001 1010 0001 1111

Step iv: Total bytes cache can store

- Cache lines = 32
- Each line holds 1 block = 8 bytes
- Total cache size = $32 \times 8 = \mathbf{256 \text{ bytes}}$

5(b) Booth's Multiplication Algorithm

Given:

- **Multiplicand (Q = -17)**
- **Multiplier (M = 16)**
- **Use 8-bit representation for Q and M**

Step 1: Convert to binary (2's complement)

1. **Q = -17 $\rightarrow$ 8-bit two's complement**
 - +17 = 00010001
 - Invert bits: 11101110
 - Add 1: 11101111  $\rightarrow$ Q = 11101111
2. **M = 16 $\rightarrow$ binary 8-bit $\rightarrow$ 00010000**

Step 2: Initialize registers

Register Initial Value

A 00000000

Q 11101111

Q-1 0

Count 8

Step 3: Booth's Algorithm Rules

- Check Q0 Q-1:
 1. 01 $\rightarrow A = A + M$
 2. 10 $\rightarrow A = A - M$
 3. 00 or 11 $\rightarrow$ do nothing
 - After operation $\rightarrow$ Right Arithmetic Shift of [A Q Q-1]
-

Step 4: Stepwise Execution (8 cycles)

Notation: [A | Q | Q-1]

Initial:

A = 00000000

Q = 11101111

Q-1 = 0

[00000000 11101111 0]

Cycle 1:

- Q0 Q-1 = 1 0 $\rightarrow A = A - M = 0 - 16 = -16 \rightarrow 11110000$ (8-bit)
 - Right arithmetic shift $\rightarrow [11111000 01110111 1]$
-

Cycle 2:

- Q0 Q-1 = 1 1 $\rightarrow$ do nothing
 - Right arithmetic shift $\rightarrow [11111100 00111011 1]$
-

Cycle 3:

- Q0 Q-1 = 1 1 $\rightarrow$ do nothing

- Right arithmetic shift $\rightarrow$ [11111110 00011101 1]
-

Cycle 4:

- $Q_0 Q_{-1} = 1\ 1 \rightarrow$ do nothing
 - Right arithmetic shift $\rightarrow$ [11111111 00001110 1]
-

Cycle 5:

- $Q_0 Q_{-1} = 0\ 1 \rightarrow A = A + M = 11111111 + 00010000 = 00001111$ (overflow ignored, 8-bit)
 - Right arithmetic shift $\rightarrow$ [00000111 10000111 0]
-

Cycle 6:

- $Q_0 Q_{-1} = 1\ 0 \rightarrow A = A - M = 00000111 - 00010000 = 11110111$
 - Right arithmetic shift $\rightarrow$ [11111011 11000011 1]
-

Cycle 7:

- $Q_0 Q_{-1} = 1\ 1 \rightarrow$ do nothing
 - Right arithmetic shift $\rightarrow$ [11111101 11100001 1]
-

Cycle 8:

- $Q_0 Q_{-1} = 1\ 1 \rightarrow$ do nothing
 - Right arithmetic shift $\rightarrow$ [11111110 11110000 1]
-

✅ Step 5: Final Result

- $[A\ Q] = 11111110\ 11110000 \rightarrow$ 16-bit signed
 - Decimal = -272 ✅
-

Shortcut / Quick Check:

- $Q = -17, M = 16 \rightarrow -17 \times 16 = -272$ ✅
-

Exam Tip:

1. Draw A, Q, Q-1 table

2. Show operation (+M or –M) at each step
 3. Show arithmetic right shift clearly
 4. Final [A Q] → product
-

5(c)(i) Access Time & Cycle Time

1. Access Time
 - Time to read/write a word from memory **once**
 - Example: 60 ns
2. Cycle Time
 - Minimum time **between starting two successive memory operations**
 - Includes access + recovery time

Access time $\leq$ Cycle time

5(c)(ii) Memory Access Types

Type	Explanation / Example
------	-----------------------

Sequential	Memory accessed one after another in order (tape). Slow.
------------	---

Direct	Memory can jump directly to a block without going sequentially (CD).
--------	--

Random	Any word can be accessed directly using address (RAM). Fastest.
--------	--

✅ Quick Summary / Exam Tip:

- Cache → divide address: Tag | Line | Byte
- Total cache bytes = lines $\times$ block size
- Booth → check Q0 Q-1 bits, shift & add/sub
- Memory access → sequential/direct/random

ঠিক আছে, চলো **Question 6** step by step সহজভাবে solve করি, বাংলা-English mix এ। 😊

6(a)(i) Responsibility of Memory Management Unit (MMU)

Definition:

👉 MMU হল CPU-এর অংশ যা virtual memory এবং physical memory-এর মধ্যে mapping করে।

Responsibilities / কাজগুলো:

1. Address Translation:

- Virtual address (program-এ ব্যবহৃত) → Physical address (RAM location)
- CPU শুধুমাত্র virtual address generate করে, MMU সেটা physical address-এ convert করে।

2. Memory Protection:

- Program একাধিক memory region-এ access করতে পারে, MMU নিশ্চিত করে illegal access না হয়।
- Example: User program memory overflow বা OS memory access control।

3. Paging / Segmentation Support:

- Virtual memory management এর জন্য page table / segment table maintain করে।

4. Cache Control (Optional):

- কিছু MMU cacheable / non-cacheable memory region control করে।

6(a)(ii) Instruction Pipeline Timing Diagram

Given:

- Pipeline stages:
 - FI = Fetch Instruction
 - DA = Decode Instruction & calculate address
 - FO = Fetch Operand
 - EX = Execute
- 9 instructions (I1 to I9)
- I5 = branch taken
- No data dependencies

Timing Diagram (simplified):

Cycle →	1	2	3	4	5	6	7	8	9	10	11	12
I1	FI	DA	FO	EX								
I2		FI	DA	FO	EX							

Cycle →	1	2	3	4	5	6	7	8	9	10	11	12
I3			FI	DA	FO	EX						
I4				FI	DA	FO	EX					
I5 (branch)					FI	DA	FO	EX				
I6						FI	DA	FO	EX			
I7							FI	DA	FO	EX		
I8								FI	DA	FO	EX	
I9									FI	DA	FO	EX

💡 **Note:** Branch taken → I6 starts from the branch target after EX of I5. No stalls due to no data dependency.

6(b) Structure of SRAM and Features

SRAM Structure:

- Static RAM uses **flip-flop circuits** to store each bit.
- Typical cell = 6 transistors (no capacitor needed).
- Stored bit stays as long as power on

Features of SRAM:

1. Fast (high speed)
2. Volatile (power off → data lost)
3. Expensive per bit
4. Used in **CPU cache (L1, L2)**

Features of DRAM:

1. Dynamic RAM uses **1 transistor + 1 capacitor** per bit
 2. Slower than SRAM
 3. Needs **periodic refresh**
 4. Cheaper and high-density → Used as main memory
-

6(c) Virtual & Physical Address, Page Miss

Definitions:

- **Virtual Address:** CPU-generated address by program, may not directly map to RAM.
- **Physical Address:** Actual address in main memory (RAM).

Page Miss Mechanism (Virtual Memory):

1. CPU requests virtual address → MMU translates to physical address
2. If page not in main memory → **Page Miss / Page Fault** occurs
3. OS interrupts CPU → loads the required page from **secondary storage (disk)** to RAM
4. Page table updated → CPU retries instruction

💡 Easy Example:

- Program wants byte 0x1F234 (virtual)
- Not in RAM → page miss → OS brings page 0x1F000–0x1F3FF into RAM → instruction executes